

LAPORAN PRAKTIKUM 8
STRUKTUR DATA
IMPLEMENTASI ABSTRACT DATA TYPE (ADT)
SORTING MENGGUNAKAN JAVA



Disusun Oleh:

Silva Nurifavan

2511533004

Dosen Pengampu:

Wahyudi. Dr. S.T.M.T

Asisten Praktikum:

Sherly Sukmadira

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Segala puji dan syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa. Berkat limpahan rahmat serta karunia yang telah Dia berikan, akhirnya laporan praktikum dengan judul “Laporan Praktikum 8 Struktur Data: Implementasi Algoritma Sorting Menggunakan Bahasa Java” ini dapat kami selesaikan dengan hasil yang baik.

Laporan ini kami susun guna memenuhi salah satu persyaratan tugas pada mata kuliah Struktur Data. Kegiatan praktikum ini bertujuan agar mahasiswa memahami prinsip dasar algoritma pengurutan data, serta mampu menerapkannya ke dalam kode program menggunakan bahasa pemrograman Java. Selain itu, mahasiswa juga diharapkan dapat mengerti mekanisme kerja dari berbagai metode pengurutan seperti Bubble Sort, Merge Sort, Quick Sort, dan Shell Sort, serta mampu membedakan sifat-sifat dan tingkat efisiensi yang dimiliki oleh masing-masing algoritma tersebut.

Penulis sepenuhnya menyadari bahwa laporan yang telah disusun ini masih jauh dari sempurna, baik dari segi tata bahasa maupun isi pembahasannya. Oleh karena itu, kami sangat mengharapkan adanya masukan berupa kritik yang membangun dan saran perbaikan, agar di kemudian hari laporan serupa dapat disajikan dengan lebih baik lagi.

Sebagai penutup, besar harapan kami semoga laporan ini dapat memberikan manfaat yang nyata, serta menambah wawasan bagi para pembaca terkait materi struktur data, khususnya dalam hal penerapan algoritma pengurutan menggunakan bahasa pemrograman Java.

Padang, 5 Juni 2026

Penulis

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman, kemampuan mengolah data dengan cepat dan efisien menjadi hal yang sangat krusial. Salah satu metode pengolahan data yang paling umum digunakan adalah algoritma pengurutan atau *sorting*. Metode ini berfungsi untuk menyusun ulang data berdasarkan aturan tertentu, misalnya dari nilai terkecil ke terbesar (menaik) atau sebaliknya dari nilai terbesar ke terkecil (menurun). Data yang sudah tersusun rapi akan sangat memudahkan langkah selanjutnya, seperti proses pencarian, pengelolaan, hingga analisis informasi yang ada.

Terdapat beragam jenis algoritma pengurutan yang bisa diterapkan, antara lain Bubble Sort, Merge Sort, Quick Sort, dan Shell Sort. Masing-masing memiliki cara kerja yang khas. Bubble Sort mengurutkan data dengan cara membandingkan pasangan elemen yang bersebelahan, lalu menukar posisinya jika urutannya belum tepat. Sementara itu, Merge Sort menerapkan konsep *divide and conquer* atau pembagian masalah, yaitu memecah kumpulan data menjadi bagian-bagian kecil, mengurutkannya satu per satu, lalu menggabungkannya kembali hingga menjadi satu kesatuan yang teratur.

Selain itu, ada juga Quick Sort yang juga menggunakan prinsip pemecahan masalah, namun memiliki perbedaan cara kerja karena bergantung pada satu elemen kunci yang disebut *pivot* sebagai patokan pemisahan data. Sedangkan Shell Sort dapat dikatakan sebagai penyempurnaan dari metode Insertion Sort, di mana proses pengurutan dilakukan dengan membandingkan elemen yang memiliki jarak tertentu, sehingga prosesnya berjalan lebih efektif dibandingkan metode dasar.

Setiap algoritma tersebut memiliki karakteristik, kelebihan, serta tingkat efisiensi yang berbeda satu sama lain. Oleh sebab itu, memahami cara kerja masing-masing metode sangatlah penting agar kita bisa memilih algoritma yang paling sesuai dengan kebutuhan pengolahan data yang sedang dikerjakan. Lebih jauh lagi, penerapan algoritma ini menggunakan bahasa pemrograman Java membantu mahasiswa memahami konsep secara lebih mendalam, baik saat membuat program sederhana berbasis teks maupun program yang memiliki tampilan antarmuka grafis atau *Graphical User Interface* (GUI).

Pada kegiatan praktikum ini, dilakukan penerapan empat jenis algoritma pengurutan, yaitu Bubble Sort, Merge Sort, Quick Sort, dan Shell Sort dengan menggunakan bahasa pemrograman Java. Kegiatan ini disusun dengan tujuan untuk memperdalam pemahaman

mahasiswa mengenai prinsip dasar pengurutan data, sekaligus melatih kemampuan dalam menerapkan algoritma tersebut agar berjalan secara efektif dan efisien.

1.2 Tujuan

1. Memahami konsep dasar dari algoritma pengurutan data (*sorting*).
2. Memahami mekanisme kerja dari metode Bubble Sort dengan tampilan antarmuka grafis, serta metode Merge Sort, Quick Sort, dan Shell Sort.
3. Mampu menerapkan atau mengimplementasikan algoritma pengurutan ke dalam kode program menggunakan bahasa pemrograman Java.
4. Mampu membandingkan cara kerja, metode, serta tingkat efisiensi yang dimiliki oleh setiap algoritma pengurutan.
5. Melatih keterampilan pemrograman dalam mengelola dan menyusun data secara efisien, serta mempraktikkan pengelolaan data menggunakan konsep struktur data.

1.3 Manfaat

1. Menambah wawasan dan pemahaman mahasiswa mengenai algoritma pengurutan data.
2. Membantu mahasiswa mengenali perbedaan mendasar dari cara kerja metode Bubble Sort berbasis antarmuka grafis, Merge Sort, Quick Sort, dan Shell Sort.
3. Melatih kemampuan mahasiswa dalam menerapkan dan menuliskan algoritma ke dalam bahasa pemrograman Java.
4. Memberikan pengalaman langsung dalam merancang dan membuat aplikasi yang memiliki tampilan antarmuka grafis.
5. Menjadi dasar pengetahuan yang kuat untuk mempelajari materi algoritma dan struktur data yang lebih kompleks di tingkat selanjutnya.

BAB II

PRMBAHASAN

Dalam ilmu pemrograman, algoritma didefinisikan sebagai serangkaian langkah sistematis dan logis yang disusun untuk menyelesaikan suatu permasalahan tertentu. Salah satu jenis algoritma yang paling sering diterapkan dalam proses pengolahan data adalah algoritma pengurutan atau *sorting*. Algoritma ini berfungsi untuk menyusun kembali sekumpulan data ke dalam urutan yang diinginkan, misalnya urutan dari nilai terkecil ke terbesar atau sebaliknya, agar data tersebut lebih mudah dikelola, diproses, dan dianalisis informasinya.

Pada kegiatan praktikum ini, dipelajari dan diterapkan empat metode pengurutan yang berbeda, yaitu Bubble Sort, Merge Sort, Quick Sort, dan Shell Sort. Keempat metode ini memiliki ciri khas dan mekanisme kerja yang berbeda satu sama lain dalam menyusun data. Penerapan atau implementasi dari keempat algoritma tersebut dilakukan menggunakan bahasa pemrograman Java, yang diwujudkan dalam dua bentuk program, yaitu program sederhana yang berjalan di layar perintah (*console*) maupun program yang dilengkapi dengan tampilan antarmuka grafis (*Graphical User Interface / GUI*).

Dalam bahasa pemrograman Java, struktur data berupa larik atau *array* digunakan sebagai wadah utama untuk menyimpan sekumpulan nilai atau data yang nantinya akan diolah. Larik ini berfungsi menampung data yang kemudian akan diproses menggunakan salah satu algoritma pengurutan yang dipilih. Setiap algoritma memiliki tahapan dan cara pemrosesan yang khas dalam mengubah susunan data di dalam larik tersebut, berikut adalah pembahasan lengkapnya:

1. Merge Sort

Merge Sort adalah metode pengurutan yang menerapkan pendekatan pemecahan masalah atau *Divide and Conquer*. Prinsip kerjanya adalah membagi keseluruhan data menjadi beberapa bagian yang lebih kecil, kemudian mengurutkan setiap bagian tersebut secara terpisah, dan akhirnya menggabungkan kembali bagian-bagian yang sudah terurut menjadi satu kesatuan data yang utuh dan tersusun rapi. Proses ini berjalan secara berulang-ulang (rekursif) hingga seluruh elemen data berada pada posisi yang benar.

Langkah-langkah kerja Merge Sort:

1. Memecah atau membagi larik data menjadi dua bagian yang berukuran lebih kecil.
2. Mengulangi proses pembagian tersebut terus-menerus sampai setiap bagian hanya terdiri dari satu elemen data saja.
3. Menggabungkan kembali bagian-bagian data tersebut dengan menempatkan setiap elemen pada urutan yang benar.
4. Melakukan proses penggabungan berulang kali hingga seluruh data tersusun dalam urutan yang sempurna.

Keunggulan dari algoritma ini adalah kestabilan kinerjanya dan tingkat efisiensi yang tinggi, terutama jika digunakan untuk mengolah data dalam jumlah besar. Namun, kelemahannya terletak pada kebutuhan memori tambahan yang cukup besar untuk menampung data sementara selama proses pemecahan dan penggabungan berlangsung.

2. Quick Sort

Quick Sort juga merupakan metode pengurutan yang menggunakan konsep pemecahan masalah (*Divide and Conquer*). Perbedaannya dengan metode lain adalah algoritma ini menentukan satu elemen khusus yang dijadikan sebagai patokan atau disebut *pivot*. Seluruh data kemudian dipisahkan menjadi dua kelompok, yaitu kelompok data yang nilainya lebih kecil dari *pivot* dan kelompok data yang nilainya lebih besar dari *pivot*.

Langkah-langkah kerja Quick Sort:

1. Menentukan salah satu elemen data sebagai titik acuan atau *pivot*.
2. Membandingkan seluruh elemen data lainnya dengan nilai *pivot* tersebut.
3. Memindahkan dan menempatkan elemen yang nilainya lebih kecil di sisi kiri *pivot*, sedangkan elemen yang nilainya lebih besar ditempatkan di sisi kanan.
4. Mengulangi langkah pemisahan tersebut pada setiap kelompok data yang terbentuk, hingga akhirnya seluruh elemen data tersusun dengan benar.

Keunggulan utama metode ini adalah kecepatan pemrosesannya yang tergolong cepat dan efisien. Akan tetapi, kinerjanya bisa menurun atau melambat jika pemilihan elemen yang dijadikan *pivot* tidak tepat atau kurang strategis.

3. Shell Sort

Shell Sort pada dasarnya merupakan pengembangan atau penyempurnaan dari metode pengurutan dasar yang dikenal sebagai Insertion Sort. Keunikan metode ini terletak pada penggunaan jarak pemisah atau *gap* tertentu saat membandingkan elemen data. Dengan cara ini, elemen yang posisinya berjauhan akan dibandingkan dan diurutkan lebih dulu, sehingga perpindahan nilai menjadi lebih efisien dibandingkan membandingkan elemen yang berdekatan saja.

Langkah-langkah kerja Shell Sort:

1. Menentukan nilai jarak pemisah atau *gap* yang akan digunakan pada proses awal.
2. Membandingkan dan mengurutkan pasangan elemen data yang memiliki jarak sesuai nilai *gap* tersebut.
3. Mengurangi nilai jarak pemisah secara bertahap menjadi semakin kecil.
4. Mengulangi proses perbandingan dan pengurutan sampai nilai jarak pemisah bernilai satu, dan seluruh data sudah tersusun rapi.
- 5.

Keunggulan dari metode ini adalah kecepatan prosesnya yang lebih baik jika dibandingkan dengan algoritma sederhana seperti Bubble Sort maupun Insertion Sort, khususnya saat menangani data berukuran sedang. Namun, tingkat efisiensinya sangat bergantung pada ketepatan pemilihan nilai jarak pemisah yang digunakan.

4. GUI Bubble Sort

Graphical User Interface (GUI) atau antarmuka grafis adalah bentuk tampilan visual yang memudahkan pengguna untuk berinteraksi dengan sebuah program aplikasi. Pada pelaksanaan praktikum ini, algoritma Bubble Sort diterapkan ke dalam bentuk aplikasi yang memiliki tampilan grafis menggunakan pustaka Java Swing.

Aplikasi Bubble Sort berbasis antarmuka grafis ini dirancang agar pengguna dapat dengan mudah memasukkan data, menjalankan proses pengurutan, serta melihat hasil dan tahapan pengurutan secara langsung melalui tampilan visual di layar. Beberapa komponen antarmuka yang digunakan dalam pembuatan program ini antara lain:

1. **JFrame**, berfungsi sebagai bingkai atau jendela utama tempat seluruh elemen program ditampilkan.

2. **TextField**, berfungsi sebagai kolom tempat pengguna memasukkan data yang akan diurutkan.
3. **Button**, berfungsi sebagai tombol perintah untuk menjalankan proses pengurutan sesuai algoritma Bubble Sort.
4. **TextArea**, berfungsi sebagai tempat menampilkan catatan proses maupun hasil akhir dari pengurutan data.

Dengan adanya tampilan grafis ini, proses pengurutan data menggunakan metode Bubble Sort menjadi jauh lebih menarik, interaktif, dan lebih mudah dipahami konsep kerjanya jika dibandingkan dengan program yang hanya berjalan dalam bentuk teks atau layar perintah biasa. Selain itu, pembuatan program berbasis antarmuka grafis ini juga melatih kemampuan mahasiswa dalam mengembangkan aplikasi berbasis komputer menggunakan bahasa pemrograman Java.

Melalui pelaksanaan praktikum ini, mahasiswa diharapkan dapat memahami prinsip dasar algoritma pengurutan, mengetahui perbedaan mendasar dari cara kerja metode Merge Sort, Quick Sort, Shell Sort, maupun Bubble Sort, serta mampu menerapkan keempat metode tersebut ke dalam bentuk kode program Java, baik yang berjalan di layar perintah maupun yang dilengkapi dengan tampilan antarmuka grafis

1. Langkah Kerja

1.1 Merge Sort

- **Kode Program**

```

1 package pekan8_2511533004;
2
3 public class MergeSort_2511533004 {
4
5     void Merge_2511533004(int arr_3004[], int l_3004, int m_3004, int r_3004) {
6
7         int n1_3004 = m_3004 - l_3004 + 1;
8         int n2_3004 = r_3004 - m_3004;
9
10        int L_3004[] = new int[n1_3004];
11        int R_3004[] = new int[n2_3004];
12
13        for (int i_3004 = 0; i_3004 < n1_3004; ++i_3004)
14            L_3004[i_3004] = arr_3004[l_3004 + i_3004];
15
16        for (int j_3004 = 0; j_3004 < n2_3004; ++j_3004)
17            R_3004[j_3004] = arr_3004[m_3004 + 1 + j_3004];
18
19        int i_3004 = 0;
20        int j_3004 = 0;
21        int k_3004 = l_3004;
22
23        while (i_3004 < n1_3004 && j_3004 < n2_3004) {
24            if (L_3004[i_3004] <= R_3004[j_3004]) {
25                arr_3004[k_3004] = L_3004[i_3004];
26                i_3004++;
27            } else {
28                arr_3004[k_3004] = R_3004[j_3004];
29                j_3004++;
30            }
31            k_3004++;
32        }
33
34        while (i_3004 < n1_3004) {
35            arr_3004[k_3004] = L_3004[i_3004];
36            i_3004++;
37            k_3004++;
38        }
39
40        while (j_3004 < n2_3004) {
41            arr_3004[k_3004] = R_3004[j_3004];
42            j_3004++;
43            k_3004++;
44        }
45    }
46
47    void sort_2511533004(int arr_3004[], int l_3004, int r_3004) {
48        if (l_3004 < r_3004) {
49            int m_3004 = (l_3004 + r_3004) / 2;
50
51            sort_2511533004(arr_3004, l_3004, m_3004);
52            sort_2511533004(arr_3004, m_3004 + 1, r_3004);
53
54            Merge_2511533004(arr_3004, l_3004, m_3004, r_3004);
55        }
56    }
57
58    static void printArray_2511533004(int arr_3004[]) {
59        int n_3004 = arr_3004.length;
60
61        for (int i_3004 = 0; i_3004 < n_3004; ++i_3004)
62            System.out.print(arr_3004[i_3004] + " ");
63
64        System.out.println();
65    }
66
67    public static void main(String args[]) {
68
69        int arr_3004[] = {12, 11, 13, 5, 6, 7};
70
71        System.out.println("Sebelum terurut");
72        printArray_2511533004(arr_3004);
73
74        MergeSort_2511533004 ob_3004 = new MergeSort_2511533004();
75        // ob_3004.sort_2511533004(arr_3004, 0, arr_3004.length - 1);
76
77        System.out.println("\nSesudah Terurut menggunakan Merge Sort");
78        printArray_2511533004(arr_3004);
79    }
80 }
81

```

Pembahasan

Program ini dibuat dengan mendefinisikan class MergeSort_2511533004 yang digunakan untuk mengimplementasikan algoritma Merge Sort pada array bertipe integer. Class ini berfungsi untuk melakukan proses pengurutan data dari nilai terkecil ke terbesar (ascending) menggunakan metode Divide and Conquer.

1. Class ini memiliki method `Merge_2511533004(int arr_3004[], int l_3004, int m_3004, int r_3004)` yang digunakan untuk menggabungkan dua bagian array yang telah terurut menjadi satu array yang tetap terurut. Method ini menerima parameter berupa array, indeks kiri, indeks tengah, dan indeks kanan.
2. Pada method `Merge_2511533004()`, variabel `n1_3004` dan `n2_3004` digunakan untuk menentukan jumlah elemen pada sub-array kiri dan sub-array kanan. Kemudian dibuat array sementara `L_3004[]` dan `R_3004[]` untuk menyimpan data dari kedua bagian array yang akan digabungkan.
3. Proses penyalinan data ke array sementara dilakukan menggunakan perulangan `for`. Array `L_3004[]` digunakan untuk menyimpan elemen dari bagian kiri, sedangkan array `R_3004[]` digunakan untuk menyimpan elemen dari bagian kanan.
4. Variabel `i_3004`, `j_3004`, dan `k_3004` digunakan sebagai indeks penelusuran. Variabel `i_3004` digunakan untuk menelusuri array kiri, `j_3004` untuk menelusuri array kanan, dan `k_3004` untuk menentukan posisi penyimpanan hasil penggabungan pada array utama.
5. Proses penggabungan dilakukan menggunakan perulangan `while`. Program akan membandingkan elemen pada array kiri dan array kanan. Nilai yang lebih kecil akan ditempatkan ke array utama terlebih dahulu sehingga urutan data tetap terjaga secara ascending.
6. Setelah salah satu array habis diproses, perulangan `while` berikutnya digunakan untuk menyalin sisa elemen yang masih terdapat pada array kiri atau array kanan ke dalam array utama sampai seluruh elemen selesai digabungkan.
7. Class ini juga memiliki method `sort_2511533004()` yang berfungsi untuk membagi array menjadi beberapa bagian yang lebih kecil secara rekursif. Method ini akan terus membagi array hingga setiap bagian hanya berisi satu elemen, kemudian memanggil method `Merge_2511533004()` untuk menggabungkannya kembali secara terurut.
8. Method `printArray_2511533004()` digunakan untuk menampilkan isi array sebelum dan sesudah proses pengurutan sehingga hasil Merge Sort dapat diamati dengan jelas.
9. Pada method `main()`, program membuat sebuah array integer yang berisi data yang belum terurut. Data tersebut kemudian ditampilkan,

diurutkan menggunakan method `sort_2511533004()`, dan hasil pengurutan ditampilkan kembali menggunakan method `printArray_2511533004()`

10. Secara keseluruhan, program ini digunakan untuk memahami cara kerja algoritma Merge Sort dalam mengurutkan data integer menggunakan metode Divide and Conquer, yaitu membagi array menjadi bagian-bagian kecil, mengurutkannya, lalu menggabungkannya kembali hingga seluruh data tersusun secara ascending.

Output

```
<terminated> MergeSort_2511533004 [Java Application] C:
Sebelum terurut
12 11 13 5 6 7

Sesudah Terurut menggunakan Merge Sort
5 6 7 11 12 13
```

1.2 Quick Sort

- **Code Program**

```
1 package pekan8_2511533004;
2
3 public class QuickSort_2511533004 {
4
5     static void swap(int[] arr_3004, int i_3004, int j_3004) {
6         int temp_3004 = arr_3004[i_3004];
7         arr_3004[i_3004] = arr_3004[j_3004];
8         arr_3004[j_3004] = temp_3004;
9     }
10
11     // Metode tambahan untuk mengatur pivot menggunakan median-of-Three
12     static void medianOfThree(int[] arr_3004, int low_3004, int high_3004) {
13         int mid_3004 = low_3004 + (high_3004 - low_3004) / 2;
14
15         // Urutkan elemen low, mid, dan high
16         if (arr_3004[low_3004] > arr_3004[mid_3004]) {
17             swap(arr_3004, low_3004, mid_3004);
18         }
19         if (arr_3004[low_3004] > arr_3004[high_3004]) {
20             swap(arr_3004, low_3004, high_3004);
21         }
22         if (arr_3004[mid_3004] > arr_3004[high_3004]) {
23             swap(arr_3004, mid_3004, high_3004);
24         }
25         swap(arr_3004, mid_3004, high_3004);
26     }
27
28     static int partition_2511533004(int[] arr_3004, int low_3004, int high_3004) {
29         // Panggil fungsi medianOfThree sebelum menentukan pivot
30         medianOfThree(arr_3004, low_3004, high_3004);
31
32         int pivot_3004 = arr_3004[high_3004]; // sekarang arr[high] sudah berisi nilai median
33         int i_3004 = (low_3004 - 1);
34
35         for (int j_3004 = low_3004; j_3004 <= high_3004 - 1; j_3004++) {
36             // jika elemen saat ini lebih kecil dari atau sama dengan pivot
37             if (arr_3004[j_3004] < pivot_3004) {
38
```

```

39         if (arr_3004[j_3004] < pivot_3004) {
40             // increment ketika elemen kecil ke kiri
41             i_3004++;
42             swap(arr_3004, i_3004, j_3004);
43         }
44         swap(arr_3004, i_3004 + 1, high_3004);
45         return (i_3004 + 1);
46     }
47
48     static void quickSort_2511533004(int[] arr_3004, int low_3004, int high_3004) {
49         if (low_3004 < high_3004) {
50             int pi_3004 = partition_2511533004(arr_3004, low_3004, high_3004);
51
52             quickSort_2511533004(arr_3004, low_3004, pi_3004 - 1);
53             quickSort_2511533004(arr_3004, pi_3004 + 1, high_3004);
54         }
55     }
56
57     public static void printArr_2511533004(int[] arr_3004) {
58         for (int i_3004 = 0; i_3004 < arr_3004.length; i_3004++) {
59             System.out.print(arr_3004[i_3004] + " ");
60         }
61         System.out.println();
62     }
63
64     public static void main(String[] args) {
65         int[] arr_3004 = {10, 7, 8, 9, 1, 5};
66         int N_3004 = arr_3004.length;
67
68         System.out.print("Data sebelum diurutkan: ");
69         printArr_2511533004(arr_3004);
70
71         quickSort_2511533004(arr_3004, 0, N_3004 - 1);
72
73         System.out.print("Data Terurut quicksort: ");
74         printArr_2511533004(arr_3004);
75     }
76 }

```

Pembahasan

Program ini dibuat dengan mendefinisikan class QuickSort_2511533004 yang digunakan untuk mengimplementasikan algoritma Quick Sort pada array bertipe integer. Class ini berfungsi untuk melakukan proses pengurutan data dari nilai terkecil ke terbesar (ascending) menggunakan metode Divide and Conquer.

1. Class ini memiliki method swap() yang digunakan untuk menukar posisi dua elemen pada array. Method ini membantu proses pengurutan ketika terdapat elemen yang harus dipindahkan ke posisi yang sesuai.
2. Method medianOfThree() digunakan untuk menentukan nilai pivot menggunakan teknik Median of Three, yaitu dengan membandingkan elemen pertama (low_3004), elemen tengah (mid_3004), dan elemen terakhir (high_3004). Teknik ini digunakan agar pemilihan pivot lebih optimal sehingga performa Quick Sort menjadi lebih baik.
3. Setelah nilai median diperoleh, elemen median dipindahkan ke posisi terakhir array menggunakan method swap(). Nilai tersebut kemudian digunakan sebagai pivot pada proses partisi.
4. Setelah nilai median diperoleh, elemen median dipindahkan ke posisi terakhir array menggunakan method swap(). Nilai tersebut kemudian digunakan sebagai pivot pada proses partisi.
5. Pada method partition_2511533004(), variabel i_3004 digunakan untuk menandai posisi elemen yang lebih kecil dari pivot, sedangkan variabel j_3004 digunakan untuk menelusuri seluruh elemen array yang akan dibandingkan dengan pivot.
6. Jika ditemukan elemen yang nilainya lebih kecil dari pivot, maka indeks i_3004 akan bertambah dan elemen tersebut ditukar menggunakan

method swap(). Setelah seluruh elemen diperiksa, pivot ditempatkan pada posisi yang benar di dalam array.

7. Method quickSort_2511533004() digunakan untuk menjalankan proses pengurutan secara rekursif. Setelah mendapatkan posisi pivot dari method partition_2511533004(), program akan mengurutkan bagian kiri dan bagian kanan array hingga seluruh data terurut.
8. Method printArr_2511533004() digunakan untuk menampilkan isi array sebelum dan sesudah proses pengurutan sehingga hasil Quick Sort dapat dilihat dengan jelas.
9. Pada method main(), program membuat sebuah array integer yang berisi data yang belum terurut. Data tersebut kemudian ditampilkan, diurutkan menggunakan method quickSort_2511533004(), dan hasil pengurutan ditampilkan kembali menggunakan method printArr_2511533004().
10. Secara keseluruhan, program ini digunakan untuk memahami cara kerja algoritma Quick Sort dengan teknik Median of Three Pivot, yaitu memilih nilai tengah dari tiga elemen sebagai pivot, kemudian membagi array menjadi beberapa bagian dan mengurutkannya secara rekursif hingga seluruh data tersusun secara ascending.

Ouput

```
<terminated> QuickSort_2511533004 [Java Application]
Data sebelum diurutkan: 10 7 8 9 1 5
Data Terurut quicksort: 1 5 7 8 9 10
```

1.3 Shell Sort

- **Kode Program**

```

1 package pekan8_2511533004;
2
3 public class ShellSort_2511533004 {
4
5     public static void shellSort_3004(int[] A_3004) {
6         int n_3004 = A_3004.length;
7         int gap_3004 = n_3004 / 2;
8
9         while (gap_3004 > 0) {
10             for (int i_3004 = gap_3004; i_3004 < n_3004; i_3004++) {
11                 int temp_3004 = A_3004[i_3004];
12                 int j_3004 = i_3004;
13
14                 while (j_3004 >= gap_3004 && A_3004[j_3004 - gap_3004] > temp_3004) {
15                     A_3004[j_3004] = A_3004[j_3004 - gap_3004];
16                     j_3004 = j_3004 - gap_3004;
17                 }
18
19                 A_3004[j_3004] = temp_3004;
20             }
21             gap_3004 = gap_3004 / 2;
22         }
23     }
24
25     public static void main(String[] args) {
26         int[] data_3004 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};
27
28         System.out.print("Sebelum: ");
29         printArray(data_3004);
30         shellSort_3004(data_3004);
31
32         System.out.print("Sesudah (Shell Sort): ");
33         printArray(data_3004);
34     }
35
36     public static void printArray(int[] arr) {
37         for (int i : arr) System.out.print(i + " ");
38         System.out.println();
39     }
40 }

```

Pembahasan

Program ini dibuat dengan mendefinisikan class QuickSort_2511533004 yang digunakan untuk mengimplementasikan algoritma Quick Sort pada array bertipe integer. Class ini berfungsi untuk melakukan proses pengurutan data dari nilai terkecil ke terbesar (ascending) menggunakan metode Divide and Conquer dengan teknik Median of Three Pivot.

1. Class ini memiliki method swap() yang digunakan untuk menukar posisi dua elemen pada array. Method ini membantu proses pengurutan ketika terdapat elemen yang harus dipindahkan ke posisi yang sesuai.
2. Method medianOfThree() digunakan untuk menentukan nilai pivot dengan membandingkan tiga elemen, yaitu elemen pertama (low_3004), elemen tengah (mid_3004), dan elemen terakhir (high_3004). Setelah dibandingkan, nilai median dipindahkan ke posisi terakhir array dan digunakan sebagai pivot.
3. Method medianOfThree() digunakan untuk menentukan nilai pivot dengan membandingkan tiga elemen, yaitu elemen pertama (low_3004), elemen tengah (mid_3004), dan elemen terakhir (high_3004). Setelah dibandingkan, nilai median dipindahkan ke posisi terakhir array dan digunakan sebagai pivot.
4. Method medianOfThree() digunakan untuk menentukan nilai pivot dengan membandingkan tiga elemen, yaitu elemen pertama (low_3004), elemen tengah (mid_3004), dan elemen terakhir (high_3004). Setelah dibandingkan, nilai median dipindahkan ke posisi terakhir array dan digunakan sebagai pivot.

5. Pada method `partition_2511533004()`, variabel `pivot_3004` digunakan untuk menyimpan nilai pivot, variabel `i_3004` digunakan sebagai penanda posisi elemen yang lebih kecil dari pivot, dan variabel `j_3004` digunakan untuk menelusuri seluruh elemen array yang akan dibandingkan.
6. Selama proses perulangan `for`, setiap elemen dibandingkan dengan pivot. Jika nilai elemen lebih kecil dari pivot, maka indeks `i_3004` akan bertambah dan posisi elemen ditukar menggunakan method `swap()`. Setelah seluruh elemen diperiksa, pivot ditempatkan pada posisi yang benar sehingga menghasilkan dua bagian array yang siap diurutkan.
7. Method `quickSort_2511533004()` digunakan untuk menjalankan proses pengurutan secara rekursif. Setelah mendapatkan posisi pivot dari method `partition_2511533004()`, program akan memanggil dirinya sendiri untuk mengurutkan bagian kiri dan bagian kanan array hingga seluruh data terurut.
8. Method `printArr_2511533004()` digunakan untuk menampilkan isi array sebelum dan sesudah proses pengurutan sehingga hasil Quick Sort dapat diamati dengan jelas.
9. Pada method `main()`, program membuat sebuah array integer yang berisi data yang belum terurut. Data tersebut kemudian ditampilkan, diurutkan menggunakan method `quickSort_2511533004()`, dan hasil pengurutan ditampilkan kembali untuk menunjukkan perubahan data setelah proses sorting.
10. Secara keseluruhan, program ini digunakan untuk memahami cara kerja algoritma Quick Sort dengan teknik Median of Three Pivot, yaitu memilih nilai median dari tiga elemen sebagai pivot, membagi array menjadi dua bagian, kemudian mengurutkannya secara rekursif hingga seluruh data tersusun secara ascending.

Output

```
<terminated> ShellSort_2511533004 [Java Application] C:\Users\
Sebelum: 3 10 4 6 8 9 7 2 1 5
Sesudah (Shell Sort): 1 2 3 4 5 6 7 8 9 10
```

1.4 Bubble Sort GUI

- **Code Program**

```
1 package pekan8_2511533004;
2
3 import java.awt.BorderLayout;
4 import java.awt.Color;
5 import java.awt.Dimension;
6 import java.awt.EventQueue;
7 import java.awt.FlowLayout;
8 import java.awt.Font;
9
10 import javax.swing.BorderFactory;
11 import javax.swing.JButton;
12 import javax.swing.JFrame;
13 import javax.swing.JLabel;
14 import javax.swing.JOptionPane;
15 import javax.swing.JPanel;
16 import javax.swing.JScrollPane;
17 import javax.swing.JTextArea;
18 import javax.swing.JTextField;
19 import javax.swing.SwingConstants;
20 import javax.swing.border.EmptyBorder;
21
22 public class BubleSortGUI_2511533004 extends JFrame {}
23
24     private static final long serialVersionUID = 1L;
25
26     private JPanel contentPane;
27     private JTextField inputField;
28     private JButton inputButton;
29     private JButton stepButton;
30     private JButton resetButton;
31     private JPanel panelArray;
32     private JTextArea stepArea;
33
34     private int[] array;
35     private JLabel[] labelArray;
36
37     private int i = 0;
38     private int j = 0;
39     private int stepCount = 1;
40     private boolean sorting = false;
41
42     public static void main(String[] args) {
43         EventQueue.invokeLater(() -> {
44             try {
45                 BubleSortGUI_2511533004 frame = new BubleSortGUI_2511533004();
46                 frame.setVisible(true);
47             } catch (Exception e) {
48                 e.printStackTrace();
49             }
50         });
51     }
52
53     public BubleSortGUI_2511533004() {
54         setTitle("Visualisasi Bubble Sort - 2511533004");
55         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
56         setBounds(100, 100, 900, 500);
57
58         contentPane = new JPanel();
59         contentPane.setBorder(new EmptyBorder(10, 10, 10, 10));
60         contentPane.setLayout(new BorderLayout(10, 10));
61         setContentPane(contentPane);
62
63         JPanel topPanel = new JPanel(new FlowLayout());
64
65         JLabel lblInput = new JLabel("Masukkan angka (pisahkan koma):");
66
67         inputField = new JTextField(25);
68
69         inputButton = new JButton("Tampilkan");
70
71         topPanel.add(inputField);
72         topPanel.add(inputButton);
73
74         contentPane.add(topPanel, BorderLayout.NORTH);
75
76         panelArray = new JPanel();
77         panelArray.setLayout(new FlowLayout());
78
79         contentPane.add(panelArray, BorderLayout.CENTER);
80
81         stepArea = new JTextArea();
82         stepArea.setEditable(false);
83
84         JScrollPane scrollPane = new JScrollPane(stepArea);
85
86         scrollPane.setPreferredSize(new Dimension(300, 0));
87
88         contentPane.add(scrollPane, BorderLayout.EAST);
89
90         JPanel bottomPanel = new JPanel();
91
92         stepButton = new JButton("Sorting Selanjutnya");
93
94         resetButton = new JButton("Reset");
95
96         stepButton.setEnabled(false);
97
98         bottomPanel.add(stepButton);
99         bottomPanel.add(resetButton);
100
101         contentPane.add(bottomPanel, BorderLayout.SOUTH);
102
103         inputButton.addActionListener(e -> setArrayFromInput());
104
105         stepButton.addActionListener(e -> nextStep());
106
107
108
```

```

109         resetButton.addActionListener(e -> reset());
110     }
111
112     private void setArrayFromInput() {
113         String text = inputField.getText().trim();
114
115         if (text.isEmpty())
116             return;
117
118         String[] parts = text.split(",");
119
120         array = new int[parts.length];
121
122         try {
123             for (int k = 0; k < parts.length; k++) {
124                 array[k] = Integer.parseInt(parts[k].trim());
125             }
126         } catch (NumberFormatException e) {
127             JOptionPane.showMessageDialog(
128                 this,
129                 "Masukkan hanya angka yang dipisahkan koma!",
130                 "Error",
131                 JOptionPane.ERROR_MESSAGE);
132             return;
133         }
134
135         i = 0;
136         j = 0;
137         stepCount = 1;
138         sorting = true;
139
140         stepButton.setEnabled(true);
141         stepArea.setText("");
142         panelArray.removeAll();
143         labelArray = new JLabel[array.length];
144
145         for (int k = 0; k < array.length; k++) {
146             labelArray[k] = new JLabel(String.valueOf(array[k]));
147             labelArray[k].setFont(new Font("Arial", Font.BOLD, 24));
148             labelArray[k].setOpaque(true);
149             labelArray[k].setBackground(Color.WHITE);
150             labelArray[k].setBorder(BorderFactory.createLineBorder(Color.BLACK));
151             labelArray[k].setPreferredSize(new Dimension(60, 50));
152             labelArray[k].setHorizontalAlignment(SwingConstants.CENTER);
153             panelArray.add(labelArray[k]);
154         }
155
156         panelArray.revalidate();
157         panelArray.repaint();
158     }
159
160     private void nextStep() {
161         if (!sorting)
162             return;
163
164         resetColors();
165
166         if (j < array.length - i - 1) {
167             labelArray[j].setBackground(Color.YELLOW);
168             labelArray[j + 1].setBackground(Color.YELLOW);
169
170             int nilaiKiri = array[j];
171             int nilaiKanan = array[j + 1];
172
173             if (array[j] > array[j + 1]) {
174                 int temp = array[j];
175                 array[j] = array[j + 1];
176                 array[j + 1] = temp;
177
178                 updateLabels();
179
180                 labelArray[j].setBackground(Color.RED);
181                 labelArray[j + 1].setBackground(Color.RED);
182
183                 stepArea.append(
184                     "Langkah " + stepCount +
185                     " : Tukar " + nilaiKiri + " dengan " + nilaiKanan + "\n");
186             } else {
187                 stepArea.append(
188                     "Langkah " + stepCount +
189                     " : Tidak ada pertukaran (" + nilaiKiri + " dan " + nilaiKanan + ")\n");
190             }
191
192             j++;
193         } else {
194             labelArray[array.length - i - 1].setBackground(Color.GREEN);
195             j = 0;
196             i++;
197         }
198
199         stepCount++;
200
201         if (i >= array.length - 1) {
202             sorting = false;
203
204             for (JLabel lbl : labelArray) {
205                 lbl.setBackground(Color.GREEN);
206             }
207
208             stepButton.setEnabled(false);
209         }

```

```

218         stepArea.append("\n=== SORTING SELESAI ===\n");
219         JOptionPane.showMessageDialog(this, "Sorting selesai!");
220     }
221 }
222
223 private void updateLabels() {
224     for (int k = 0; k < array.length; k++) {
225         labelArray[k].setText(String.valueOf(array[k]));
226     }
227 }
228
229 private void resetColors() {
230     if (labelArray == null)
231         return;
232
233     for (JLabel lbl : labelArray) {
234         if (!Color.GREEN.equals(lbl.getBackground())) {
235             lbl.setBackground(Color.WHITE);
236         }
237     }
238 }
239
240 private void reset() {
241     inputField.setText("");
242     panelArray.removeAll();
243     panelArray.revalidate();
244     panelArray.repaint();
245     stepArea.setText("");
246     stepButton.setEnabled(false);
247     array = null;
248     labelArray = null;
249     sorting = false;
250     i = 0;
251     j = 0;
252     stepCount = 1;
253 }
254 }

```

Pembahasan

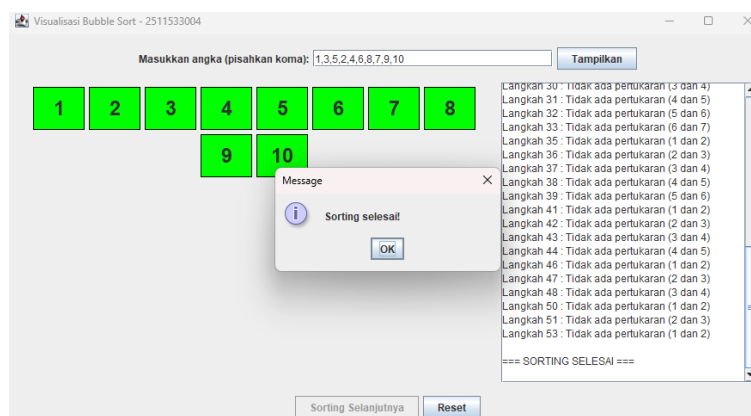
Program ini dibuat dengan mendefinisikan class BubbleSortGUI_2511533004 yang digunakan untuk mengimplementasikan algoritma Bubble Sort dalam bentuk Graphical User Interface (GUI) menggunakan Java Swing. Program ini memungkinkan pengguna memasukkan data, melihat proses pengurutan langkah demi langkah, serta menampilkan hasil pengurutan secara visual.

1. Class ini merupakan turunan dari JFrame yang berfungsi sebagai jendela utama aplikasi. Berbagai komponen GUI seperti JTextField, JButton, JPanel, JLabel, dan JTextArea digunakan untuk membangun tampilan program.
2. Method konstruktor BubbleSortGUI_2511533004() digunakan untuk mengatur tampilan antarmuka, seperti membuat panel input, panel visualisasi array, area penjelasan langkah sorting, serta tombol untuk menjalankan proses sorting dan reset data.
3. Method setArrayFromInput_2511533004() digunakan untuk membaca data yang dimasukkan pengguna melalui JTextField. Data yang dipisahkan dengan tanda koma akan diubah menjadi array integer dan ditampilkan dalam bentuk label pada panel visualisasi.
4. Pada proses visualisasi, setiap elemen array ditampilkan menggunakan JLabel. Label tersebut digunakan untuk menunjukkan nilai data yang sedang diproses selama algoritma Bubble Sort berlangsung.
5. Method nextStep_2511533004() digunakan untuk menjalankan satu langkah proses Bubble Sort setiap kali tombol Sorting Selanjutnya ditekan. Program akan membandingkan dua elemen yang berdekatan dan menukarnya jika urutannya salah.
6. Selama proses pengurutan, warna label digunakan untuk memperjelas visualisasi. Warna kuning menunjukkan elemen yang sedang dibandingkan, warna merah menunjukkan elemen yang ditukar,

sedangkan warna hijau menunjukkan elemen yang sudah berada pada posisi yang benar.

7. Method `updateLabels_2511533004()` digunakan untuk memperbarui tampilan nilai pada label setelah terjadi pertukaran elemen. Dengan demikian pengguna dapat melihat perubahan data secara langsung pada layer.
8. Method `resetColors_2511533004()` digunakan untuk mengembalikan warna label ke kondisi awal sebelum proses perbandingan berikutnya dilakukan, sehingga visualisasi menjadi lebih jelas dan mudah dipahami.
9. Method `reset_2511533004()` digunakan untuk menghapus seluruh data yang telah dimasukkan, membersihkan area visualisasi, mengosongkan riwayat langkah sorting, serta mengembalikan program ke kondisi awal.
10. Pada method `main()`, program dijalankan menggunakan `EventQueue.invokeLater()` untuk menampilkan jendela GUI. Pengguna kemudian dapat memasukkan data, menjalankan Bubble Sort secara bertahap, melihat setiap proses perbandingan dan pertukaran data, serta mengamati hasil pengurutan hingga selesai.
11. Secara keseluruhan, program ini digunakan untuk memahami cara kerja algoritma Bubble Sort secara visual. Melalui GUI, pengguna dapat melihat setiap langkah perbandingan dan pertukaran elemen secara langsung sehingga proses pengurutan data menjadi lebih interaktif dan mudah dipahami dibandingkan tampilan berbasis console.

Output



TUGAS PRATIKUM 8

Algoritma Sorting (Quick Sort)

Saya memilih algoritma Quick Sort karena algoritma ini memiliki proses pengurutan yang cepat, efisien, dan menggunakan konsep rekursi yang menarik untuk dipelajari. Selain itu, Quick Sort merupakan salah satu algoritma sorting yang banyak digunakan dalam pemrograman karena performanya yang baik. Dengan mengimplementasikan Quick Sort, saya dapat memahami cara kerja pivot, partition, dan rekursi dalam proses pengurutan data. Oleh karena itu, algoritma ini dipilih sebagai implementasi pada praktikum pengurutan playlist lagu.

Class Lagu_2511533004

Kode Program

```
1 package pekan8_2511533004;
2 public class Lagu_2511533004 {
3     String judul_3004;
4     String penyanyi_3004;
5     int durasi_3004;
6
7     public Lagu_2511533004(String judul_3004, String penyanyi_3004, int durasi_3004) {
8         this.judul_3004 = judul_3004;
9         this.penyanyi_3004 = penyanyi_3004;
10        this.durasi_3004 = durasi_3004;
11    }
12 }
```

Pembahasan

Program ini dibuat dengan mendefinisikan class Lagu_2511533004 yang digunakan sebagai representasi data lagu pada program playlist. Class ini berfungsi untuk menyimpan informasi setiap lagu yang nantinya akan diurutkan menggunakan algoritma Quick Sort.

1. Class ini memiliki atribut judul_3004 bertipe String untuk menyimpan judul lagu, atribut penyanyi_3004 bertipe String untuk menyimpan nama penyanyi, dan atribut durasi_3004 bertipe int untuk menyimpan lama durasi lagu dalam satuan detik.
2. Program menggunakan konstruktor Lagu_2511533004(String judul_3004, String penyanyi_3004, int durasi_3004) untuk menginisialisasi objek lagu. Konstruktor ini akan menerima data judul, penyanyi, dan durasi, kemudian menyimpannya ke dalam atribut class menggunakan keyword this.
3. Setiap objek yang dibuat dari class ini merepresentasikan satu data lagu yang akan disimpan ke dalam array dataLagu_3004 pada class Sorting_2511533004.
4. Secara keseluruhan, class Lagu_2511533004 berfungsi sebagai struktur data sederhana yang digunakan untuk menyimpan informasi lagu sebelum dilakukan proses pengurutan menggunakan algoritma Quick Sort berdasarkan durasi lagu secara ascending (dari durasi terpendek ke terpanjang).

Class Sorting_2511533004

Kode Program

```

1 package pekan8_2511533004;
2
3 import java.util.Scanner;
4
5 public class Sorting_2511533004 {
6     Lagu_2511533004[] dataLagu_3004 = new Lagu_2511533004[20];
7     int jumlahData_3004 = 0;
8
9     // Input data lagu
10    public void inputData_3004() {
11        Scanner input_3004 = new Scanner(System.in);
12
13        System.out.print("Jumlah lagu (minimal 7): ");
14        jumlahData_3004 = input_3004.nextInt();
15        input_3004.nextLine();
16
17        for (int i_3004 = 0; i_3004 < jumlahData_3004; i_3004++) {
18            System.out.println("\nData lagu ke-" + (i_3004 + 1));
19
20            System.out.print("Judul : ");
21            String judul_3004 = input_3004.nextLine();
22
23            System.out.print("Penyanyi : ");
24            String penyanyi_3004 = input_3004.nextLine();
25
26            System.out.print("Durasi (detik) : ");
27            int durasi_3004 = input_3004.nextInt();
28            input_3004.nextLine();
29
30            dataLagu_3004[i_3004] = new Lagu_2511533004(
31                judul_3004,
32                penyanyi_3004,
33                durasi_3004
34            );
35        }
36    }
37
38    // Menukar data lagu
39    public void swap_3004(int i_3004, int j_3004) {
40        Lagu_2511533004 temp_3004 = dataLagu_3004[i_3004];
41        dataLagu_3004[i_3004] = dataLagu_3004[j_3004];
42        dataLagu_3004[j_3004] = temp_3004;
43    }
44
45    // Partisi untuk Quick Sort
46    public int partition_3004(int low_3004, int high_3004) {
47        int pivot_3004 = dataLagu_3004[high_3004].durasi_3004;
48
49        int i_3004 = low_3004 - 1;
50
51        for (int j_3004 = low_3004; j_3004 < high_3004; j_3004++) {
52            if (dataLagu_3004[j_3004].durasi_3004 < pivot_3004) {
53                i_3004++;
54                swap_3004(i_3004, j_3004);
55            }
56        }
57        swap_3004(i_3004 + 1, high_3004);
58        return i_3004 + 1;
59    }
60
61    // Metode Quick Sort
62    public void quickSort_3004(int low_3004, int high_3004) {
63        if (low_3004 < high_3004) {
64            int pi_3004 = partition_3004(low_3004, high_3004);
65
66            quickSort_3004(low_3004, pi_3004 - 1);
67            quickSort_3004(pi_3004 + 1, high_3004);
68        }
69    }
70
71    // Menampilkan data lagu
72    public void tampilkanData_3004() {
73        for (int i_3004 = 0; i_3004 < jumlahData_3004; i_3004++) {
74            System.out.println("Judul: " + dataLagu_3004[i_3004].judul_3004 +
75                " | Penyanyi: " + dataLagu_3004[i_3004].penyanyi_3004 +
76                " | Durasi: " + dataLagu_3004[i_3004].durasi_3004 + " detik");
77        }
78    }
79
80    // Method utama program
81    public static void main(String[] args) {
82        Sorting_2511533004 playlist_3004 = new Sorting_2511533004();
83
84        System.out.println("== Sorting Playlist NIM: 2511533004 ==");
85
86        playlist_3004.inputData_3004();
87
88        System.out.println("\n== Data Sebelum Sorting ==");
89        playlist_3004.tampilkanData_3004();
90
91        playlist_3004.quickSort_3004(0, playlist_3004.jumlahData_3004 - 1);
92
93        System.out.println("\n=== Data Setelah Quick Sort (Durasi Asc) ===");
94        playlist_3004.tampilkanData_3004();
95    }
96 }
97

```

Pembahasan

Sorting_2511533004 yang digunakan untuk mengelola data playlist lagu serta mengimplementasikan algoritma Quick Sort untuk mengurutkan lagu berdasarkan durasi dari yang terpendek hingga terpanjang.

1. Class ini memiliki atribut `dataLagu_3004` berupa array objek `Lagu_2511533004` dengan kapasitas 20 data yang digunakan untuk menyimpan informasi lagu. Selain itu terdapat atribut `jumlahData_3004` yang digunakan untuk menyimpan jumlah lagu yang dimasukkan oleh pengguna.
2. Method `inputData_3004()` digunakan untuk memasukkan data lagu melalui keyboard menggunakan class `Scanner`. Pada method ini pengguna diminta memasukkan judul lagu, nama penyanyi, dan durasi lagu, kemudian data tersebut disimpan ke dalam array `dataLagu_3004`.
3. Method `swap_3004()` digunakan untuk menukar posisi dua objek lagu dalam array. Method ini berperan penting dalam proses pengurutan karena digunakan saat terjadi pertukaran data selama algoritma Quick Sort berjalan.
4. Method `partition_3004()` digunakan untuk membagi data berdasarkan nilai pivot. Pada program ini pivot diambil dari durasi lagu pada elemen terakhir array. Selanjutnya setiap data dibandingkan dengan pivot untuk menentukan apakah harus ditempatkan di sebelah kiri atau kanan pivot.
5. Method `quickSort_3004()` merupakan implementasi utama algoritma Quick Sort. Method ini bekerja secara rekursif dengan memanggil method `partition_3004()` untuk membagi array menjadi beberapa bagian yang lebih kecil, kemudian mengurutkan masing-masing bagian hingga seluruh data tersusun dengan benar berdasarkan durasi lagu secara ascending.
6. Method `tampilData_3004()` digunakan untuk menampilkan seluruh data lagu yang tersimpan pada array. Informasi yang ditampilkan meliputi judul lagu, penyanyi, dan durasi lagu sehingga pengguna dapat melihat kondisi data sebelum maupun sesudah proses sorting.
7. Pada method `main()`, program diawali dengan membuat objek `Sorting_2511533004`, kemudian pengguna diminta memasukkan data lagu. Setelah itu program menampilkan data sebelum sorting, menjalankan algoritma Quick Sort, dan menampilkan kembali data yang telah terurut berdasarkan durasi lagu dari yang terkecil ke terbesar.
8. Secara keseluruhan, program ini digunakan untuk memahami cara kerja algoritma Quick Sort dalam mengurutkan data objek pada array. Melalui program ini dapat dipahami proses `partition`, pertukaran data, rekursi, serta penerapan algoritma sorting dalam pengelolaan playlist lagu secara terstruktur dan efisien.

Output

```

k:terminated> Sorting_2511533004 [Java Application] C:\Users\Lenovo\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64.jdk\bin\java.exe
== Sorting Playlist NIM: 2511533004 ==
Jumlah lagu (minimal 7): 7

Data lagu ke-1
Judul : suka kamu
Penyanyi : silva
Durasi (detik) : 350

Data lagu ke-2
Judul : luka
Penyanyi : silva
Durasi (detik) : 240

Data lagu ke-3
Judul : mama
Penyanyi : silva
Durasi (detik) : 140

Data lagu ke-4
Judul : monokrom
Penyanyi : tulus
Durasi (detik) : 389

Data lagu ke-5
Judul : lalala
Penyanyi : silva
Durasi (detik) : 220

Data lagu ke-6
Judul : evaluasi
Penyanyi : hindia
Durasi (detik) : 305

Data lagu ke-7
Judul : komang
Penyanyi : raim laode
Durasi (detik) : 540

== Data Sebelum Sorting ==
Judul: suka kamu | Penyanyi: silva | Durasi: 350 detik
Judul: luka | Penyanyi: silva | Durasi: 240 detik
Judul: mama | Penyanyi: silva | Durasi: 140 detik
Judul: monokrom | Penyanyi: tulus | Durasi: 389 detik
Judul: lalala | Penyanyi: silva | Durasi: 220 detik
Judul: evaluasi | Penyanyi: hindia | Durasi: 305 detik
Judul: komang | Penyanyi: raim laode | Durasi: 540 detik

==== Data Setelah Quick Sort (Durasi Asc) ====
Judul: mama | Penyanyi: silva | Durasi: 140 detik
Judul: lalala | Penyanyi: silva | Durasi: 220 detik
Judul: luka | Penyanyi: silva | Durasi: 240 detik
Judul: evaluasi | Penyanyi: hindia | Durasi: 305 detik
Judul: suka kamu | Penyanyi: silva | Durasi: 350 detik
Judul: monokrom | Penyanyi: tulus | Durasi: 389 detik
Judul: komang | Penyanyi: raim laode | Durasi: 540 detik

```